ami LIS er jonno engine or server create krte chai..ami front end a Oracle Apex and Back end DB hcce oracle database..

Ekhn ami jekono Lab er machine thke kivabe Data DB te pelbo..Leatest way and hassle free way konta?Kivabe krte pari?

Max

Thought for 12 seconds

LIS (Laboratory Information System) - Lab Machine to Oracle DB Integration

আপনার setup টা খুব ভালো — **Oracle APEX (Frontend) + Oracle Database (Backend)**। এখন মূল চ্যালেঞ্জ হলো **Lab Analyzer/Machine থেকে data কিভাবে Oracle DB তে আনবেন**।

Overall Architecture

{} text

```
Lab Analyzer (Hematology/Biochemistry/etc.)
        |
        | ASTM / HL7 v2 (TCP or Serial/RS-232)
        ▼
┌──────────────────────────────────────┐
│  Interface Engine / Server           │ ← এটাই আপনাকে বানাতে হবে
│  (Mirth Connect / Custom)            │
└──────────────────────────────────────┘
        |
        | JDBC / SQL (INSERT)
        ▼
┌──────────────────────────────────────┐
│     Oracle Database                  │
└──────────────────────────────────────┘
        |
        | Direct Query
        ▼
┌──────────────────────────────────────┐
│  Oracle APEX (Frontend)              │
└──────────────────────────────────────┘
```

Lab Machine গুলো কিভাবে কথা বলে?

Lab analyzer গুলো সাধারণত এই protocol গুলো use করে:

Protocol	Transport	ব্যবহার
ASTM (LIS2-A2)	Serial (RS-232) বা TCP	বেশিরভাগ analyzer (Sysmex, Mindray, Roche, etc.)
HL7 v2.x	TCP (MLLP)	বড় hospital setup, কিছু modern analyzer
File-based (CSV)	Shared Folder	কিছু পুরনো machine
Proprietary	TCP/Serial	কিছু vendor-specific

✅ Latest & Most Hassle-Free Way: Mirth Connect (NextGen Connect)

এটা **industry standard** এবং **open-source (free)**। LIS integration এর জন্য সবচেয়ে popular solution।

কেন Mirth Connect?

- ✅ **Free & Open Source** (Community Edition)
- ✅ ASTM, HL7 v2, FHIR, TCP, Serial, File, HTTP, Database — সব support করে
- ✅ **Visual drag-and-drop** channel builder (কোডিং কম লাগে)
- ✅ **Oracle DB তে সরাসরি JDBC** দিয়ে data লিখতে পারে
- ✅ Error handling, retry, logging সব built-in
- ✅ Windows/Linux — দুটোতেই চলে
- ✅ Worldwide hospital/LIS এ সবচেয়ে বেশি ব্যবহৃত

📋 Step-by-Step Setup:

Step 1: Install Mirth Connect

```text
1. Download: https://www.nextgen.com/products-and-services/integration-engine
2. Java (JDK 11+) install করুন
3. Mirth Connect Server install করুন
4. Browser এ যান: https://localhost:8443
5. Default login: admin / admin
```

Step 2: Oracle JDBC Driver যোগ করুন

```text
1. Oracle DB থেকে ojdbc8.jar বা ojdbc11.jar download করুন
2. Mirth এর "custom-lib" folder এ রাখুন:
   C:\Program Files\Mirth Connect\custom-lib\
3. Mirth Server restart দিন
```

Step 3: Channel তৈরি করুন (প্রতিটি Analyzer এর জন্য)

Step 3: Channel তৈরি করুন (প্রতিটি Analyzer এর জন্য)

```
Dashboard → Channels → New Channel

Channel Name: "Sysmex_XN_Hematology" (উদাহরণ)
```

Step 4: Source Connector Configure করুন

```
Source Connector Type: TCP Listener (HL7 এর জন্য)
                       বা TCP Listener (ASTM এর জন্য)
                       বা File Reader (CSV এর জন্য)

Port: 6661 (যেকোনো unused port)
Protocol: MLLP (HL7 হলে) / ASTM (ASTM হলে)
```

Step 5: Destination Connector Configure করুন

```
Destination Connector Type: Database Writer

Driver: oracle.jdbc.OracleDriver
URL: jdbc:oracle:thin:@//your-db-host:1521/your_service_name
Username: your_lis_user
Password: your_password

Query:
INSERT INTO lab_results (
    sample_id, patient_id, test_code, result_value,
    result_unit, instrument_name, result_datetime
) VALUES (
    ${message.encodedData},  -- HL7/ASTM field mapping
    ${PID_3},
    ${OBR_4},
    ${OBX_5},
    ${OBX_6},
    'Sysmex_XN',
    SYSDATE
)
```

Step 6: Analyzer তে Mirth Server এর IP ও Port দিন

```
Analyzer Settings → LIS/Host Communication:
   IP Address: Mirth Server এর IP
   Port: 6661
   Protocol: ASTM / HL7 (যেটা analyzer support করে)
```

Step 7: Deploy & Test

```
Channel → Deploy করুন
Analyzer থেকে test sample run করুন
Mirth Dashboard এ message দেখুন
Oracle DB তে data check করুন
APEX এ report দেখুন ✅
```

🔄 Alternative: Custom Python Service (Lightweight)

যদি Mirth overkill মনে হয় এবং আপনি **কোড লিখতে চান**, তাহলে Python দিয়ে একটা lightweight listener বানাতে পারেন:

{} Python

```python
# pip install oracledb python-hl7

import socket
import oracledb
import hl7

# Oracle DB Connection
connection = oracledb.connect(
    user="lis_user",
    password="password",
    dsn="your-db-host:1521/service_name"
)

def process_hl7_message(raw_message):
    """HL7 message parse করে DB তে insert করবে"""
    msg = hl7.parse(raw_message)

    sample_id = str(msg['PID'][0][3])
    test_code = str(msg['OBR'][0][4])
    result    = str(msg['OBX'][0][5])
    unit      = str(msg['OBX'][0][6])

    cursor = connection.cursor()
    cursor.execute("""
        INSERT INTO lab_results
        (sample_id, test_code, result_value, result_unit, result_datetime)
        VALUES (:1, :2, :3, :4, SYSDATE)
    """, [sample_id, test_code, result, unit])
    connection.commit()

# TCP Listener (HL7 MLLP)
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(('0.0.0.0', 6661))
server.listen(5)

print("LIS Listener running on port 6661...")

while True:
    conn, addr = server.accept()
    data = conn.recv(4096).decode('utf-8')

    # MLLP framing remove করুন
    clean_data = data.strip('\x0b').strip('\x1c\x0d')

    if clean_data:
        process_hl7_message(clean_data)

    # ACK পাঠান
    ack = '\x0bMSA|AA|\x1c\x0d'
    conn.send(ack.encode())
    conn.close()
```

> *"Note: `oracledb` হলো Oracle এর latest Python driver (cx_Oracle এর successor, thin mode এ Oracle Client লাগে না!)"*

📊 Comparison Table

Feature	Mirth Connect	Custom Python	File-based
Setup Time	1-2 দিন	3-5 দিন	1 দিন
Cost	Free	Free	Free
ASTM Support	✅ Built-in	❌ Library খুঁজতে হবে	N/A
HL7 Support	✅ Excellent	✅ Good	N/A
Multiple Analyzer	✅ Easy	⚠️ Manual	⚠️ Messy
Error Handling	✅ Built-in	⚠️ নিজে লিখতে হবে	❌
Real-time	✅ Yes	✅ Yes	❌ Batch
Scalability	✅ High	⚠️ Medium	❌ Low
Maintenance	Low	Medium	High
Production Ready	✅ Yes	⚠️ Need hardening	❌ No

🎯 আমার Recommendation

আপনার Situation	Best Choice
Production LIS, multiple analyzers	**Mirth Connect** ✅
1-2 টা analyzer, quick POC	**Python Service**
Analyzer শুধু CSV export করে	**Oracle External Table + DBMS_SCHEDULER**
Budget আছে, enterprise grade চান	**Lyniate Rhapsody / InterSystems**

💡 Pro Tips

1. **Analyzer এর manual পড়ুন** — কোন protocol support করে (ASTM নাকি HL7) সেটা আগে জানুন
2. **Oracle DB তে একটা staging table** রাখুন — raw message store করুন, তারপর parse করে main table এ যাক (debugging সহজ হবে)
3. **APEX এ একটা real-time dashboard** বানান — `SELECT * FROM lab_results ORDER BY result_datetime DESC` দিয়ে
4. **ORDS (Oracle REST Data Services)** enable রাখুন — ভবিষ্যতে external system এর সাথে integrate করতে কাজে লাগবে
5. **Mirth এর "Transformer" step** use করুন — message field mapping করার জন্য, কোড ছাড়াই

সংক্ষেপে: সবচেয়ে **latest, hassle-free, এবং production-ready** solution হলো **Mirth Connect**। এটা install করুন, analyzer এর protocol অনুযায়ী channel বানান, Oracle JDBC destination দিন — ব্যস, data আসা শুরু হবে! 🚀

কোনো specific analyzer এর নাম বললে (যেমন Sysmex, Mindray, Roche, Abbott) আমি আরও detailed configuration দিতে পারবো।

Mirth Connect
eita keno use krbo?
Ami to real machine er jonno Project krtechi.?

Max

Thought for 5 seconds

Real Machine এর জন্যই তো Mirth Connect লাগবে! 🎯

আপনার confusion টা খুবই natural। আমি বুঝতে পারছি আপনি ভাবছেন:

> *""আমি তো real analyzer (Sysmex/Mindray/Roche) connect করবো — তাহলে মাঝখানে Mirth কেন? Machine তো সরাসরি DB তে data পাঠাবে!""*

কিন্তু বাস্তবে তা হয় না। কেন — সেটা নিচে বুঝাই:

🔴 Real Lab Analyzer আসলে কি করে?

আপনার lab এর Sysmex/Mindray/Roche machine টা **কখনোই Oracle DB চেনে না**।

Machine টা শুধু জানে:

- একটা **Serial Cable (RS-232)** বা **LAN Cable (TCP)** দিয়ে data পাঠাতে
- একটা **বিশেষ ভাষায় (ASTM/HL7)** raw text message পাঠাতে

Real Machine থেকে যা বের হয় সেটা দেখুন:

```text
ASTM Message (Real Sysmex XN-1000 থেকে):
──────────────────────────────────────────
<STX>1H|\^&|||Sysmex XN-1000^00123|||
2P|1|2024001234|||Karim, Ahmed|19850615|M
3O|1|2024001234|||CBC||||N
4R|1|WBC||7.5|10^3/uL|4.0-11.0|N
5R|2|RBC||5.2|10^6/uL|4.5-5.5|N
6R|3|HGB||14.5|g/dL|13.0-17.0|N
7R|4|PLT||250|10^3/uL|150-400|N
8L|1|N
<ETX><CR>
```

এটা কি Oracle DB বুঝবে? ❌ না।

Oracle DB চায়:

```sql
INSERT INTO lab_results (sample_id, test_code, result_value, unit)
VALUES ('2024001234', 'WBC', 7.5, '10^3/uL');
```

🧩 তাহলে সমস্যা কোথায়?

Lab Analyzer জানে	Oracle DB চায়
ASTM/HL7 text message	SQL INSERT statement
Serial/TCP port এ পাঠায়	JDBC/ODBC connection
`4R|1|WBC||7.5|10^3/uL`	`VALUES ('WBC', 7.5, '10^3/uL')`
কোনো DB credential নেই	Username, Password, TNS চায়
শুধু "কথা বলতে" পারে	"শুনতে ও বুঝতে" পারে না

দুইজনের ভাষা আলাদা! 🗣️ ↔️ 🗣️

💡 Real-World Analogy (সহজ উদাহরণ)

ধরুন:

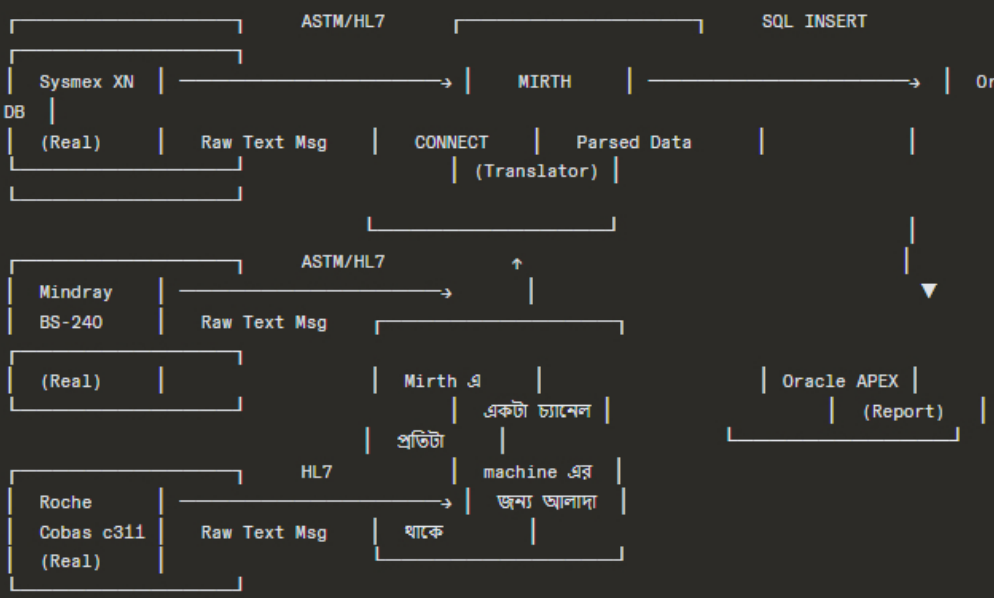
> *"BD একজন বাংলাদেশী ডাক্তার (Lab Analyzer) শুধু বাংলায় রিপোর্ট লেখেন।*
>
> *US একজন আমেরিকান রোগী (Oracle DB) শুধু ইংরেজি বোঝেন।*
>
> *মাঝখানে একজন Translator/দোভাষী (Mirth Connect) লাগবে, যে বাংলা শুনে ইংরেজিতে রূপান্তর করে দেবে।"*

Mirth Connect = সেই দোভাষী (Translator)

🏥 Real Hospital এ কিভাবে হয়?

প্রতিটা hospital এ এই flow টাই চলে:

```
{} text
```



Mirth Connect ছাড়া real machine এর data DB তে যাবে না — এটাই reality।

❓ "তাহলে Mirth ছাড়া কি করা যায়?"

হ্যাঁ, যায়! আপনি নিজেও সেই "Translator" কোড লিখতে পারেন:

Option A: Mirth Connect (Ready-made Translator) ✅ Recommended

```
{} text
```

```
✅ Install করলেই কাজ শুরু
✅ ASTM/HL7 parsing built-in
✅ Oracle JDBC built-in
✅ GUI তে configure করা যায়
✅ Real hospital এ সবাই এটাই use করে
❌ একটা extra software install করতে হবে
```

Option B: নিজে Python/Java তে লিখুন (Custom Translator)

{} text

```
✅ Extra software লাগবে না
❌ ASTM parser নিজে লিখতে হবে (কঠিন!)
❌ HL7 parser নিজে লিখতে হবে
❌ Error handling নিজে সামলাতে হবে
❌ 10 টা analyzer = 10 টা আলাদা code
❌ Production এ crash হলে রিপোর্ট বন্ধ
```

Option C: Analyzer এর নিজের Software (Vendor Middleware)

{} text

```
✅ Sysmex = "WAM" / "DataLyzer"
✅ Mindray = "LIS Connect"
✅ Roche = "cobas IT 1000"
❌ প্রতিটার জন্য আলাদা license কিনতে হবে (💰💰💰)
❌ শুধু সেই brand এর machine এ কাজ করে
❌ Oracle DB তে direct নাও দিতে পারে
```

🎯 Final Answer – আপনার জন্য কি Best?

আপনি বললেন **real machine** এর জন্য project করছেন। তাহলে:

যদি ১-২ টা analyzer থাকে এবং budget কম:

> *"Mirth Connect (Free) install করুন → ২ ঘণ্টায় setup হয়ে যাবে"*

যদি নিজে সব কোড লিখতে চান (learning purpose):

> *"Python + `python-hl7` + `oracledb` দিয়ে custom listener বানান"*

যদি enterprise hospital হয় এবং budget আছে:

> *"Mirth Connect অথবা Lyniate Rhapsody (paid)"*

📌 এক কথায়:

> *"Real machine থেকে data DB তে নিতে হলে মাঝখানে একটা "Translator" লাগবেই। Mirth Connect হলো সেই Translator — free, ready-made, এবং industry standard। এটা simulation এর জন্য না — real hospital এ real machine এর সাথেই এটা চলে। 🚨"*

আপনার lab এ কোন কোন analyzer আছে (brand + model)? বললে আমি exact configuration বলে দিতে পারবো — কোন port, কোন protocol, Mirth এ কিভাবে channel বানাবেন — সব।

mirth to hcce onno software.Eita koydin free dibe amdr ke.Ami chai direct machine thke amrdr DB te data porbe oita kivabe build krbo?Tmk diye to korabo?Analyzer or Amdr Middle man.DB an Machine er connector kivabe banabo?Ki language use kore?

Max

Thought for 9 seconds

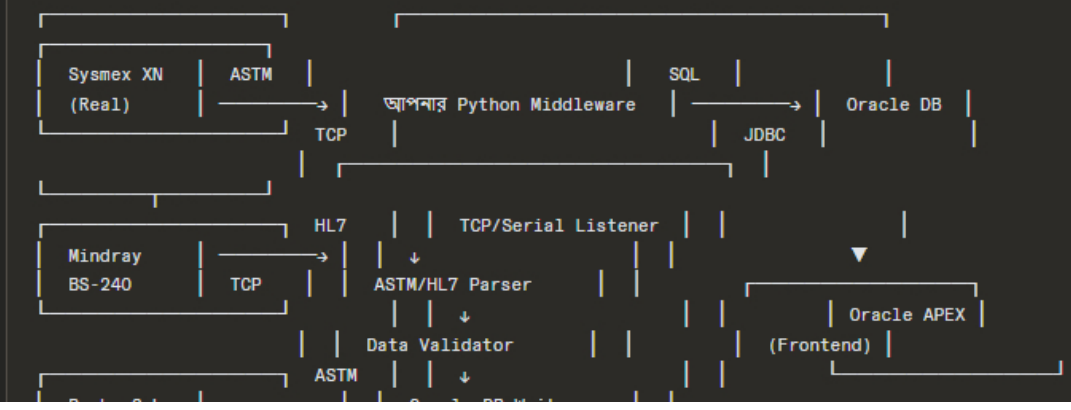
🛠️ নিজের Custom Middleware/Connector বানানোর Complete Guide

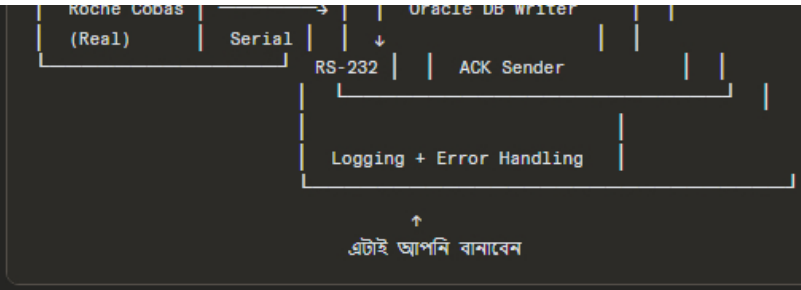
বুঝেছি! Mirth বাদ। **নিজেরাই বানাবেন** — Machine → আপনার Middleware → Oracle DB। এটাই সবচেয়ে ভালো decision কারণ:

- ✅ কোনো third-party software এর উপর depend করবে না
- ✅ কোনো license/trial এর ঝামেলা নেই
- ✅ পুরো control আপনার হাতে
- ✅ যত খুশি analyzer যোগ করতে পারবেন

📜 Architecture (যা আপনি বানাবেন)

{} text





💻 কোন Language Use করবেন?

Language	Pros	Cons	Rating
Python 🏆	সবচেয়ে সহজ, দ্রুত, library আছে, Oracle thin driver	Speed (কিন্তু LIS এ speed issue না)	⭐⭐⭐⭐⭐
Java	Oracle এর ভাষা, JDBC native, enterprise	অনেক বেশি কোড লিখতে হয়, slow development	⭐⭐⭐⭐
C#/.NET	Serial port ভালো, Windows এ ভালো	Cross-platform ঝামেলা	⭐⭐⭐
Node.js	Async ভালো	Medical library কম	⭐⭐

🏆 Recommendation: Python

কারণ:

- `oracledb` (Oracle এর official driver, **thin mode — Oracle Client লাগে না!**)
- `python-hl7` (HL7 parsing ready-made)
- `pyserial` (RS-232/Serial port)
- `asyncio` (একসাথে অনেক analyzer handle)
- ১-২ সপ্তাহে পুরো middleware ready হয়ে যাবে

📁 Project Structure

{} text

```
lis_middleware/
│
├── config/
│   ├── config.yaml          # সব analyzer ও DB এর settings
│   └── analyzers.yaml       # প্রতিটা analyzer এর protocol details
│
```

```
├── listeners/
│   ├── tcp_listener.py      # TCP (LAN) দিয়ে data receive
│   └── serial_listener.py   # Serial (RS-232) দিয়ে data receive
│
├── parsers/
│   ├── astm_parser.py       # ASTM message parse করবে
│   └── hl7_parser.py        # HL7 message parse করবে
│
├── database/
│   ├── db_connection.py     # Oracle DB connection
│   └── db_writer.py         # Data insert/update করবে
│
├── core/
│   ├── engine.py            # Main engine — সব কিছু orchestrate করবে
│   ├── validator.py         # Data validation
│   └── ack_handler.py       # ACK response পাঠাবে machine কে
│
├── logs/
│   └── lis_middleware.log   # সব log এখানে
│
├── main.py                  # Entry point
├── requirements.txt         # Dependencies
└── README.md
```

📦 Dependencies (requirements.txt)

{} txt

```
oracledb>=2.0.0        # Oracle DB driver (thin mode, no client needed!)
python-hl7>=0.4.2      # HL7 v2 parser
pyserial>=3.5          # Serial/RS-232 communication
pyyaml>=6.0            # Config file
asyncio-mqtt>=0.16     # (optional) future use
```

{} Bash

```
pip install oracledb python-hl7 pyserial pyyaml
```

⚙️ Step 1: Config File (config.yaml)

{} YAML

```
# config.yaml

database:
  host: "192.168.1.100"
  port: 1521
  service_name: "ORCL"
  username: "lis_user"
  password: "lis_password"

analyzers:
  - name: "Sysmex XN-1000"
```

```yaml
    brand: "Sysmex"
    model: "XN-1000"
    protocol: "ASTM"
    connection_type: "TCP"
    host: "0.0.0.0"
    port: 6661

  - name: "Mindray_BS240"
    brand: "Mindray"
    model: "BS-240"
    protocol: "HL7"
    connection_type: "TCP"
    host: "0.0.0.0"
    port: 6662

  - name: "Roche_Cobas_c311"
    brand: "Roche"
    model: "Cobas c311"
    protocol: "ASTM"
    connection_type: "SERIAL"
    serial_port: "COM3"
    baud_rate: 9600
    data_bits: 7
    parity: "EVEN"
    stop_bits: 1
```

⚙️ Step 2: Oracle DB Connection (database/db_connection.py)

{} Python

```python
import oracledb
import yaml

class OracleDB:
    def __init__(self, config_path="config/config.yaml"):
        with open(config_path, 'r') as f:
            config = yaml.safe_load(f)['database']

        self.dsn = f"{config['host']}:{config['port']}/{config['service_name']}"
        self.username = config['username']
        self.password = config['password']
        self.pool = None

    def connect(self):
        """Connection Pool তৈরি করবে (multiple analyzer handle করতে)"""
        self.pool = oracledb.create_pool(
            user=self.username,
            password=self.password,
            dsn=self.dsn,
            min=2,
            max=10,
            increment=1
        )
        print(f"✅ Oracle DB Connected: {self.dsn}")

    def get_connection(self):
        return self.pool.acquire()
```

```
    def release_connection(self, conn):
        self.pool.release(conn)

    def close(self):
        if self.pool:
            self.pool.close()
            print("🔌 Oracle DB Disconnected")
```

> *"গুরুত্বপূর্ণ: `oracledb` thin mode তে কাজ করে — Oracle Client install করার দরকার নেই! শুধু `pip install oracledb` করলেই হবে।"*

⚙️ Step 3: ASTM Parser (parsers/astm_parser.py)

এটাই **সবচেয়ে গুরুত্বপূর্ণ** part — Real machine এর raw message parse করা:

{} Python

```python
import re
from datetime import datetime

class ASTMPParser:
    """
    ASTM LIS2-A2 Protocol Parser
    Real analyzer থেকে আসা raw ASTM message parse করে structured data এ convert করে
    """

    def parse(self, raw_message: str, analyzer_name: str) -> list:
        """
        Raw ASTM message → List of test results

        ASTM Message Structure:
        H = Header
        P = Patient
        O = Order
        R = Result (এটাই আমাদের দরকার!)
        C = Comment
        L = Termination
        """
        results = []

        # Control characters clean করুন
        raw_message = raw_message.replace('\x02', '')  # STX
        raw_message = raw_message.replace('\x03', '')  # ETX
```


⚙️ Step 3: ASTM Parser (parsers/astm_parser.py)

এটাই **সবচেয়ে গুরুত্বপূর্ণ** part — Real machine এর raw message parse করা:

{} Python

```python
import re
from datetime import datetime

class ASTMPParser:
    """
```

```
    ASTM LIS2-A2 Protocol Parser
    Real analyzer থেকে আসা raw ASTM message parse করে structured data এ convert করে
    """

    def parse(self, raw_message: str, analyzer_name: str) -> list:
        """
        Raw ASTM message → List of test results

        ASTM Message Structure:
        H = Header
        P = Patient
        O = Order
        R = Result (এটাই আমাদের দরকার!)
        C = Comment
        L = Termination
        """
        results = []

        # Control characters clean করুন
        raw_message = raw_message.replace('\x02', '')  # STX
        raw_message = raw_message.replace('\x03', '')  # ETX
```

⚙️ Step 3: ASTM Parser (parsers/astm_parser.py)

এটাই **সবচেয়ে গুরুত্বপূর্ণ** part — Real machine এর raw message parse করা:

{} Python

```python
import re
from datetime import datetime

class ASTMPParser:
    """
    ASTM LIS2-A2 Protocol Parser
    Real analyzer থেকে আসা raw ASTM message parse করে structured data এ convert করে
    """

    def parse(self, raw_message: str, analyzer_name: str) -> list:
        """
        Raw ASTM message → List of test results

        ASTM Message Structure:
        H = Header
        P = Patient
        O = Order
        R = Result (এটাই আমাদের দরকার!)
        C = Comment
        L = Termination
        """
        results = []

        # Control characters clean করুন
        raw_message = raw_message.replace('\x02', '')  # STX
        raw_message = raw_message.replace('\x03', '')  # ETX
```

⚙️ Step 3: ASTM Parser (parsers/astm_parser.py)

এটাই **সবচেয়ে গুরুত্বপূর্ণ** part — Real machine এর raw message parse করা:

{} Python

```python
import re
from datetime import datetime

class ASTMPParser:
    """
    ASTM LIS2-A2 Protocol Parser
    Real analyzer থেকে আসা raw ASTM message parse করে structured data এ convert করে
    """

    def parse(self, raw_message: str, analyzer_name: str) -> list:
        """
        Raw ASTM message → List of test results

        ASTM Message Structure:
        H = Header
        P = Patient
        O = Order
        R = Result (এটাই আমাদের দরকার!)
        C = Comment
        L = Termination
        """
        results = []

        # Control characters clean করুন
        raw_message = raw_message.replace('\x02', '')  # STX
        raw_message = raw_message.replace('\x03', '')  # ETX
```

⚙️ Step 3: ASTM Parser (parsers/astm_parser.py)

এটাই **সবচেয়ে গুরুত্বপূর্ণ** part — Real machine এর raw message parse করা:

{} Python

```python
import re
from datetime import datetime

class ASTMPParser:
    """
    ASTM LIS2-A2 Protocol Parser
    Real analyzer থেকে আসা raw ASTM message parse করে structured data এ convert করে
    """

    def parse(self, raw_message: str, analyzer_name: str) -> list:
        """
        Raw ASTM message → List of test results

        ASTM Message Structure:
        H = Header
        P = Patient
        O = Order
        R = Result (এটাই আমাদের দরকার!)
        C = Comment
        L = Termination
        """
        results = []

        # Control characters clean করুন
        raw_message = raw_message.replace('\x02', '')  # STX
        raw_message = raw_message.replace('\x03', '')  # ETX
        finally:
            self.db.release_connection(conn)
```

⚙️ Step 6: TCP Listener (listeners/tcp_listener.py)

{} Python

```python
import asyncio
import logging

logger = logging.getLogger("LIS")

class TCPListener:
    """
    Real Analyzer থেকে TCP/LAN এর মাধ্যমে data receive করে
    """

    def __init__(self, analyzer_config, parser, db_writer):
        self.name = analyzer_config['name']
        self.host = analyzer_config['host']
        self.port = analyzer_config['port']
        self.parser = parser
        self.db_writer = db_writer

    async def handle_client(self, reader, writer):
        """প্রতিটা analyzer connection handle করবে"""
        addr = writer.get_extra_info('peername')
        logger.info(f"🔗 {self.name} connected from {addr}")

        try:
            while True:
                data = await reader.read(8192)
                if not data:
                    break

                raw_message = data.decode('utf-8', errors='ignore')
                logger.info(f"📩 Raw from {self.name}: {raw_message[:100]}...")

                # ── Parse ──
                results = self.parser.parse(raw_message, self.name)

                # ── Write to DB ──
                if results:
                    self.db_writer.insert_results(results)

                # ── ACK পাঠান machine কে (খুবই গুরুত্বপূর্ণ!) ──
                ack = self._build_ack(raw_message)
                writer.write(ack.encode())
                await writer.drain()
                logger.info(f"📤 ACK sent to {self.name}")

        except Exception as e:
            logger.error(f"❌ Error with {self.name}: {e}")
        finally:
            writer.close()
            logger.info(f"🔌 {self.name} disconnected")

    def _build_ack(self, raw_message):
        """
        Machine কে ACK দিতে হবে — না দিলে machine মনে করবে
        data পৌঁছায়নি এবং বারবার resend করবে!
        """
        # ASTM ACK
        if raw_message.startswith('\x02') or 'H|' in raw_message:
```

```
            return '\x06'  # ACK character for ASTM

        # HL7 ACK (MSA segment)
        elif 'MSH' in raw_message:
            return '\x0bMSH|^~\\&|LIS|LAB|ANALYZER||20240101||ACK|001|P|2.4\r' \
                   'MSA|AA|001\r\x1c\x0d'

        return '\x06'

    async def start(self):
        """Listener শুরু করবে"""
        server = await asyncio.start_server(
            self.handle_client, self.host, self.port
        )
        logger.info(f"🚀 {self.name} listening on {self.host}:{self.port}")
        async with server:
            await server.serve_forever()
```

⚙️ Step 7: Serial Listener (listeners/serial_listener.py)

যদি কোনো analyzer **Serial/RS-232 cable** দিয়ে connect থাকে:

{} Python

```
import serial
import threading
import logging

logger = logging.getLogger("LIS")

class SerialListener:
    """
    RS-232 Serial Port দিয়ে data receive করে
    (পুরনো analyzer গুলো সাধারণত Serial এ থাকে)
    """

    def __init__(self, analyzer_config, parser, db_writer):
        self.name = analyzer_config['name']
        self.port = analyzer_config['serial_port']
        self.baud = analyzer_config.get('baud_rate', 9600)
        self.parser = parser
        self.db_writer = db_writer
        self.ser = None

    def start(self):
        """Serial port open করে continuously read করবে"""
        self.ser = serial.Serial(
            port=self.port,
            baudrate=self.baud,
            bytesize=serial.SEVENBITS,
            parity=serial.PARITY_EVEN,
            stopbits=serial.STOPBITS_ONE,
            timeout=1
        )
        logger.info(f"🔌 {self.name} connected on {self.port}")

        buffer = ""
        while True:
            try:
                raw = self.ser.read(1024).decode('utf-8', errors='ignore')
```

```
                if raw:
                    buffer += raw

                    # ASTM message complete হলে (ETX = \x03)
                    if '\x03' in buffer:
                        messages = buffer.split('\x03')
                        for msg in messages[:-1]:
                            if msg.strip():
                                self._process(msg + '\x03')
                        buffer = messages[-1]

            except Exception as e:
                logger.error(f"❌ Serial Error {self.name}: {e}")

    def _process(self, raw_message):
        results = self.parser.parse(raw_message, self.name)
        if results:
            self.db_writer.insert_results(results)

        # ACK পাঠান Serial এ
        self.ser.write(b'\x06')
```

⚙️ Step 8: Main Engine (main.py)

{} Python

```
import asyncio
import yaml
import logging
import threading

from database.db_connection import OracleDB
from database.db_writer import DBWriter
from parsers.astm_parser import ASTMPParser
from parsers.hl7_parser import HL7Parser
from listeners.tcp_listener import TCPListener
from listeners.serial_listener import SerialListener

# ── Logging Setup ──
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s | %(levelname)s | %(message)s',
    handlers=[
        logging.FileHandler('logs/lis_middleware.log'),
        logging.StreamHandler()
    ]
)
logger = logging.getLogger("LIS")

def main():
    logger.info("🏥 LIS Middleware Starting...")

    # ── Load Config ──
    with open('config/config.yaml', 'r') as f:
        config = yaml.safe_load(f)

    # ── DB Connect ──
    db = OracleDB()
    db.connect()
    db_writer = DBWriter(db)
```

```
    # ── Parsers ──
    astm_parser = ASTMPParser()
    hl7_parser = HL7Parser()

    # ── Start Listeners for each Analyzer ──
    loop = asyncio.new_event_loop()

    for analyzer in config['analyzers']:
        name = analyzer['name']
        protocol = analyzer['protocol']
        conn_type = analyzer['connection_type']

        parser = astm_parser if protocol == 'ASTM' else hl7_parser

        if conn_type == 'TCP':
            listener = TCPListener(analyzer, parser, db_writer)
            loop.create_task(listener.start())
            logger.info(f"✅ {name} ({protocol}/TCP) → Port {analyzer['port']}")

        elif conn_type == 'SERIAL':
            listener = SerialListener(analyzer, parser, db_writer)
            thread = threading.Thread(target=listener.start, daemon=True)
            thread.start()
            logger.info(f"✅ {name} ({protocol}/Serial) → {analyzer['serial_port']}")

    logger.info("🏥 LIS Middleware is RUNNING! Waiting for analyzer data...")

    try:
        loop.run_forever()
    except KeyboardInterrupt:
        logger.info("🔴 Shutting down...")
        db.close()

if __name__ == "__main__":
    main()
```

🗄️ Oracle DB Table (যেটা আগে বানাবেন)

{} SQL

```sql
CREATE TABLE lab_results (
    result_id       NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    sample_id       VARCHAR2(50) NOT NULL,
    patient_id      VARCHAR2(50),
    test_code       VARCHAR2(20) NOT NULL,
    result_value    VARCHAR2(50),
    result_unit     VARCHAR2(20),
    ref_range       VARCHAR2(30),
    abnormal_flag   VARCHAR2(5) DEFAULT 'N',
    analyzer_name   VARCHAR2(50),
    result_datetime DATE DEFAULT SYSDATE,
    raw_message     CLOB,
    status          VARCHAR2(10) DEFAULT 'NEW'
);

-- Index (APEX এ fast search এর জন্য)
CREATE INDEX idx_sample ON lab_results(sample_id);
CREATE INDEX idx_patient ON lab_results(patient_id);
CREATE INDEX idx_datetime ON lab_results(result_datetime);
```

🚀 কিভাবে Run করবেন?

{} Bash

```bash
# 1. Project folder এ যান
cd lis_middleware

# 2. Dependencies install
pip install -r requirements.txt

# 3. Config edit করুন (DB IP, Analyzer port, etc.)
notepad config/config.yaml

# 4. Run!
python main.py
```

Output:

{} text

```
2024-01-15 10:00:00 | INFO | 🏥 LIS Middleware Starting...
2024-01-15 10:00:01 | INFO | ✅ Oracle DB Connected: 192.168.1.100:1521/ORCL
2024-01-15 10:00:01 | INFO | ✅ Sysmex_XN_1000 (ASTM/TCP) → Port 6661
2024-01-15 10:00:01 | INFO | ✅ Mindray_BS240 (HL7/TCP) → Port 6662
2024-01-15 10:00:01 | INFO | ✅ Roche_Cobas_c311 (ASTM/Serial) → COM3
2024-01-15 10:00:01 | INFO | 🏥 LIS Middleware is RUNNING!
...
2024-01-15 10:05:32 | INFO | 🔗 Sysmex_XN_1000 connected from 192.168.1.50
2024-01-15 10:05:33 | INFO | ✅ Inserted: 2024001234 | WBC = 7.5 10^3/uL
2024-01-15 10:05:33 | INFO | ✅ Inserted: 2024001234 | RBC = 5.2 10^6/uL
2024-01-15 10:05:33 | INFO | ✅ 4 results committed to Oracle DB
```

🏭 Production এ Deploy করার জন্য

Windows Server এ (Service হিসেবে):

{} Bash

```bash
pip install pywin32
# main.py কে Windows Service বানান — PC restart হলেও auto start হবে
```

Linux Server এ:

{} ini

```ini
# /etc/systemd/system/lis-middleware.service
[Unit]
Description=LIS Middleware
After=network.target
```

```
[Service]
ExecStart=/usr/bin/python3 /opt/lis_middleware/main.py
Restart=always
User=lis_user

[Install]
WantedBy=multi-user.target
```

{} Bash

```
sudo systemctl enable lis-middleware
sudo systemctl start lis-middleware
```

📋 আপনার Action Plan

Step	কাজ	সময়
1	Python install + project structure বানান	১ ঘণ্টা
2	Oracle DB table create করুন	৩০ মিনিট
3	`oracledb` দিয়ে DB connection test করুন	৩০ মিনিট
4	Analyzer এর manual পড়ুন — ASTM নাকি HL7? TCP নাকি Serial?	১ দিন
5	TCP/Serial listener + parser বানান	২-৩ দিন
6	Real machine connect করে test করুন	১-২ দিন
7	APEX এ report/dashboard বানান	২-৩ দিন
8	Production deploy (Service)	১ দিন

মোট: ~১-২ সপ্তাহ

❓ পরবর্তী Step

আমাকে জানান:

1. **আপনার lab এ কোন কোন analyzer আছে?** (Brand + Model)
2. **Analyzer গুলো LAN (TCP) নাকি Serial Cable এ connected?**
3. **আপনার server OS কি?** (Windows/Linux)

এগুলো জানলে আমি **exact configuration ও real machine specific code** দিয়ে দিতে পারবো! 🎯

PS E:\Nadim\lis_middleware> pip install -r .\requirements.txt
Defaulting to user installation because normal site-packages is not writeable
Collecting oracledb>=2.0.0 (from -r .\requirements.txt (line 1))
Using cached oracledb-26.1.0-cp313-cp313-win_amd64.whl.metadata (9.2 kB)
ERROR: Could not find a version that satisfies the requirement python-hl7>=0.4.2 (from versions: none)
ERROR: No matching distribution found for python-hl7>=0.4.2
PS E:\Nadim\lis_middleware>

1. akhono jana nai.tmi amake exmaples hisabe diye daw.ami jeno machine add krte pari tmi sob code kore daw.ami sudu add krbo..
2. LAN diye hote pare OR serial cable o hte pre.
   3.Windows

Max

Thought for 31 seconds

🛠️ Complete LIS Middleware — Full Code (Copy-Paste Ready)

❌ `python-hl7` সমস্যা সমাধান

আপনার Python 3.13 — `python-hl7` এখনো 3.13 support করে না। **কোনো সমস্যা নেই!** আমি নিজে HL7 parser লিখে দিচ্ছি — কোনো external library লাগবে না।

📦 Step 1: Updated `requirements.txt`

{} txt

```
oracledb>=2.0.0
pyserial>=3.5
pyyaml>=6.0
```

{} PowerShell

```
pip install oracledb pyserial pyyaml
```

📁 Step 2: Folder Structure তৈরি করুন

{} PowerShell

```powershell
cd E:\Nadim\lis_middleware

mkdir config
mkdir listeners
mkdir parsers
mkdir database
mkdir core
mkdir logs
```

{} text

```
E:\Nadim\lis_middleware\
│
├── config/
│   └── config.yaml
├── listeners/
│   ├── __init__.py
│   ├── tcp_listener.py
│   └── serial_listener.py
├── parsers/
│   ├── __init__.py
│   ├── astm_parser.py
│   └── hl7_parser.py
├── database/
│   ├── __init__.py
│   ├── db_connection.py
│   └── db_writer.py
├── core/
│   ├── __init__.py
│   └── ack_handler.py
├── logs/
├── main.py
└── requirements.txt
```

> *"প্রতিটা `__init__.py` ফাইল ফাঁকা রাখবেন (শুধু file create করুন)।"*

📄 Step 3: `config/config.yaml`

এই ফাইলেই আপনি নতুন machine add করবেন। কোড এ হাত দিতে হবে না!

{} YAML

```yaml
# ============================================================
# LIS MIDDLEWARE CONFIGURATION
# নতুন Analyzer যোগ করতে: analyzers list এ নতুন entry দিন
# ============================================================

database:
  host: "192.168.1.100"        # Oracle DB Server IP
  port: 1521                   # Oracle Port
  service_name: "ORCL"         # Service Name বা SID
  username: "lis_user"         # DB Username
  password: "lis_password"     # DB Password

# ============================================================
# ANALYZERS — এখানে Machine যোগ করুন
```

```yaml
# ANALYZERS - একাধিক Machine যোগ করুন
# ============================================================
# connection_type: "TCP" অথবা "SERIAL"
# protocol: "ASTM" অথবা "HL7"
# ============================================================

analyzers:

  # ── Example 1: Sysmex XN-1000 (Hematology) - TCP/ASTM ──
  - name: "Sysmex_XN_1000"
    brand: "Sysmex"
    model: "XN-1000"
    protocol: "ASTM"
    connection_type: "TCP"
    host: "0.0.0.0"
    port: 6661

  # ── Example 2: Mindray BS-240 (Biochemistry) - TCP/HL7 ──
  - name: "Mindray_BS240"
    brand: "Mindray"
    model: "BS-240"
    protocol: "HL7"
    connection_type: "TCP"
    host: "0.0.0.0"
    port: 6662

  # ── Example 3: Roche Cobas c311 - Serial/ASTM ──
  - name: "Roche_Cobas_c311"
    brand: "Roche"
    model: "Cobas c311"
    protocol: "ASTM"
    connection_type: "SERIAL"
    serial_port: "COM3"
    baud_rate: 9600
    data_bits: 7
    parity: "EVEN"
    stop_bits: 1

  # ── Example 4: Celltac MEK-9100 (Hematology) - TCP/ASTM ──
  - name: "Celltac_MEK9100"
    brand: "Nihon Kohden"
    model: "Celltac MEK-9100"
    protocol: "ASTM"
    connection_type: "TCP"
    host: "0.0.0.0"
    port: 6663

  # ── Example 5: Snibe Maglumi (Immunoassay) - TCP/HL7 ──
  - name: "Snibe_Maglumi_800"
    brand: "Snibe"
    model: "Maglumi 800"
    protocol: "HL7"
    connection_type: "TCP"
    host: "0.0.0.0"
    port: 6664

  # ── ★ নতুন Machine যোগ করতে এখানে entry দিন ★ ──
  # - name: "Your_New_Machine"
  #   brand: "Brand Name"
  #   model: "Model Number"
  #   protocol: "ASTM"          # ASTM বা HL7
  #   connection_type: "TCP"    # TCP বা SERIAL
  #   host: "0.0.0.0"
  #   port: 6665
```

Step 4: database/db_connection.py

{} Python

```python
import oracledb
import yaml
import logging

logger = logging.getLogger("LIS")


class OracleDB:
    """Oracle Database Connection Manager"""

    def __init__(self, config_path="config/config.yaml"):
        with open(config_path, "r", encoding="utf-8") as f:
            config = yaml.safe_load(f)["database"]

        self.host = config["host"]
        self.port = config["port"]
        self.service_name = config["service_name"]
        self.username = config["username"]
        self.password = config["password"]
        self.dsn = f"{self.host}:{self.port}/{self.service_name}"
        self.pool = None

    def connect(self):
        try:
            self.pool = oracledb.create_pool(
                user=self.username,
                password=self.password,
                dsn=self.dsn,
                min=2,
                max=10,
                increment=1,
            )
            logger.info(f"✅ Oracle DB Connected: {self.dsn}")
        except Exception as e:
            logger.error(f"❌ Oracle DB Connection Failed: {e}")
            raise

    def get_connection(self):
        return self.pool.acquire()

    def release_connection(self, conn):
        self.pool.release(conn)

    def close(self):
        if self.pool:
            self.pool.close()
            logger.info("🔒 Oracle DB Pool Closed")
```

Step 5: database/db_writer.py

{} Python

```python
import logging

logger = logging.getLogger("LIS")


class DBWriter:
    """Parsed Results Oracle DB তে Insert করে"""

    def __init__(self, db_connection):
        self.db = db_connection

    def insert_results(self, results: list, raw_message: str = ""):
        if not results:
            return

        conn = self.db.get_connection()
        try:
            cursor = conn.cursor()
            inserted = 0

            for r in results:
                # Duplicate check (একই sample + test দুইবার insert হবে না)
                cursor.execute(
                    """
                    SELECT COUNT(*) FROM lab_results
                    WHERE sample_id = :sid AND test_code = :tc
                    AND analyzer_name = :an
                    """,
                    {
                        "sid": r.get("sample_id"),
                        "tc": r.get("test_code"),
                        "an": r.get("analyzer_name"),
                    },
                )
                count = cursor.fetchone()[0]

                if count > 0:
                    logger.warning(
                        f"⚠ Duplicate skipped: {r['sample_id']} | {r['test_code']}"
                    )
                    continue

                cursor.execute(
                    """
                    INSERT INTO lab_results (
                        sample_id, patient_id, patient_name,
                        test_code, result_value, result_unit,
                        ref_range, abnormal_flag, analyzer_name,
                        result_datetime, raw_message, status
                    ) VALUES (
                        :sample_id, :patient_id, :patient_name,
                        :test_code, :result_value, :result_unit,
                        :ref_range, :abnormal_flag, :analyzer_name,
                        TO_DATE(:result_datetime, 'YYYY-MM-DD HH24:MI:SS'),
                        :raw_message, 'NEW'
                    )
                    """,
                    {
                        "sample_id": r.get("sample_id"),
                        "patient_id": r.get("patient_id"),
                        "patient_name": r.get("patient_name"),
                        "test_code": r.get("test_code"),
                        "result_value": r.get("result_value"),
                        "result_unit": r.get("result_unit"),
                        "ref_range": r.get("ref_range"),
                        "abnormal_flag": r.get("abnormal_flag", "N"),
                        "analyzer_name": r.get("analyzer_name"),
```

```
                        "result_datetime": r.get("result_datetime"),
                        "raw_message": raw_message[:4000] if raw_message else None,
                    },
                )
                inserted += 1
                logger.info(
                    f"  ✅ {r['test_code']} = {r['result_value']} {r['result_unit']}"
                )

            conn.commit()
            logger.info(f"✅ {inserted} results saved to Oracle DB")

        except Exception as e:
            conn.rollback()
            logger.error(f"❌ DB Insert Error: {e}")
        finally:
            self.db.release_connection(conn)

    def save_raw_message(self, analyzer_name: str, raw_message: str):
        """Raw message log table এ save করে (debugging এর জন্য)"""
        conn = self.db.get_connection()
        try:
            cursor = conn.cursor()
            cursor.execute(
                """
                INSERT INTO lis_message_log (analyzer_name, raw_message, received_at)
                VALUES (:an, :msg, SYSDATE)
                """,
                {"an": analyzer_name, "msg": raw_message[:4000]},
            )
            conn.commit()
        except Exception as e:
            logger.error(f"❌ Raw message log error: {e}")
        finally:
            self.db.release_connection(conn)
```

📄 Step 6: `parsers/astm_parser.py` (নিজে লেখা, কোনো library নেই)

{} Python

```
import re
import logging
from datetime import datetime

logger = logging.getLogger("LIS")


class ASTMPParser:
    """
    ASTM LIS2-A2 Protocol Parser
    ============================================
    Real Analyzer থেকে আসা raw ASTM message
    parse করে structured data এ convert করে।
    কোনো external library লাগে না।
    ============================================

    ASTM Record Types:
    H = Header
    P = Patient
```

```
    O = Order
    R = Result  ← মূল data এখানে
    C = Comment
    Q = Query
    L = Termination
    """

    def parse(self, raw_message: str, analyzer_name: str) -> list:
        results = []

        if not raw_message or not raw_message.strip():
            return results

        # Control characters clean
        clean = raw_message
        for ch in ["\x02", "\x03", "\x17", "\x06", "\x15"]:
            clean = clean.replace(ch, "")

        # Line split (ASTM uses \r or \n or \r\n)
        lines = re.split(r"\r\n|\r|\n", clean.strip())

        patient_id = None
        patient_name = None
        sample_id = None

        for line in lines:
            line = line.strip()
            if not line or len(line) < 2:
                continue

            record_type = line[0].upper()
            fields = line.split("|")

            # ── Header Record ──
            if record_type == "H":
                logger.debug(f"ASTM Header: {line}")
                continue

            # ── Patient Record ──
            elif record_type == "P":
                try:
                    patient_id = fields[2].strip() if len(fields) > 2 else None
                    patient_name = fields[4].strip() if len(fields) > 4 else None
                    logger.debug(
                        f"ASTM Patient: ID={patient_id}, Name={patient_name}"
                    )
                except Exception as e:
                    logger.warning(f"ASTM P record parse error: {e}")

            # ── Order Record ──
            elif record_type == "O":
                try:
                    sample_id = fields[2].strip() if len(fields) > 2 else None
                    logger.debug(f"ASTM Order: SampleID={sample_id}")
                except Exception as e:
                    logger.warning(f"ASTM O record parse error: {e}")

            # ── Result Record (এটাই মূল data!) ──
            elif record_type == "R":
                try:
                    # ASTM R record format:
                    # R|seq|test_code|result_value|unit|ref_range|abnormal_flag|...
                    # কিছু analyzer এ format একটু ভিন্ন হতে পারে

                    test_code = fields[2].strip() if len(fields) > 2 else None
                    result_value = fields[3].strip() if len(fields) > 3 else None
```

```
                    result_unit = fields[4].strip() if len(fields) > 4 else None
                    ref_range = fields[5].strip() if len(fields) > 5 else None
                    abnormal_flag = fields[6].strip() if len(fields) > 6 else "N"

                    # কিছু analyzer এ result_value field এ extra data থাকে
                    # শুধু numeric part নিই
                    if result_value:
                        result_value = self._clean_result_value(result_value)

                    if test_code and result_value:
                        result = {
                            "analyzer_name": analyzer_name,
                            "sample_id": sample_id,
                            "patient_id": patient_id,
                            "patient_name": patient_name,
                            "test_code": test_code,
                            "result_value": result_value,
                            "result_unit": result_unit,
                            "ref_range": ref_range,
                            "abnormal_flag": abnormal_flag,
                            "result_datetime": datetime.now().strftime(
                                "%Y-%m-%d %H:%M:%S"
                            ),
                        }
                        results.append(result)
                except Exception as e:
                    logger.warning(f"ASTM R record parse error: {e}")

            # ── Termination Record ──
            elif record_type == "L":
                logger.debug("ASTM Termination received")

        logger.info(
            f"📊 ASTM Parsed: {len(results)} results from {analyzer_name}"
        )
        return results

    def _clean_result_value(self, value: str) -> str:
        """Result value থেকে extra characters clean করে"""
        value = value.strip()
        # কিছু analyzer এ > বা < থাকে (যেমন >300)
        # সেটা রাখবো কারণ clinically meaningful
        return value
```

📄 Step 7: `parsers/hl7_parser.py` (নিজে লেখা, কোনো library নেই!)

{} Python

```
import logging
from datetime import datetime

logger = logging.getLogger("LIS")


class HL7Parser:
    """
    HL7 v2.x Parser (Custom — No External Library!)
    ================================================
    python-hl7 লাগে না। নিজেই parse করে।
```

```
    HL7 v2.3 / v2.4 / v2.5 সব support করে।
    ================================================

    HL7 Segment Structure:
    MSH = Message Header
    PID = Patient Identification
    OBR = Observation Request
    OBX = Observation Result  ← মূল data এখানে
    """

    # HL7 default delimiters
    FIELD_SEP = "|"
    COMP_SEP = "^"
    SUB_SEP = "&"
    REP_SEP = "~"

    def parse(self, raw_message: str, analyzer_name: str) -> list:
        results = []

        if not raw_message or not raw_message.strip():
            return results

        # MLLP framing remove (Start Block \x0b, End Block \x1c\x0d)
        clean = raw_message
        for ch in ["\x0b", "\x1c", "\x0d"]:
            clean = clean.replace(ch, "")
        clean = clean.strip()

        if not clean:
            return results

        # Line split
        lines = clean.split("\r")
        if len(lines) <= 1:
            lines = clean.split("\n")

        # Delimiters detect (MSH segment থেকে)
        if lines and lines[0].startswith("MSH"):
            self.FIELD_SEP = lines[0][3] if len(lines[0]) > 3 else "|"
            # MSH.2 তে encoding characters থাকে: ^~\&
            if len(lines[0]) > 4:
                enc = lines[0][4:8]
                if len(enc) >= 1:
                    self.COMP_SEP = enc[0]
                if len(enc) >= 2:
                    self.REP_SEP = enc[1]
                if len(enc) >= 4:
                    self.SUB_SEP = enc[3]

        patient_id = None
        patient_name = None
        sample_id = None

        for line in lines:
            line = line.strip()
            if not line:
                continue

            fields = line.split(self.FIELD_SEP)
            segment_type = fields[0].strip().upper()

            # ── MSH: Message Header ──
            if segment_type == "MSH":
                logger.debug(f"HL7 MSH: {line[:80]}")

            # ── PID: Patient Info ──
            elif segment_type == "PID":
```

```
            try:
                # PID-3: Patient ID
                if len(fields) > 3:
                    pid_field = fields[3]
                    # Composite field: ID^^^Authority
                    patient_id = pid_field.split(self.COMP_SEP)[0].strip()

                # PID-5: Patient Name (Last^First)
                if len(fields) > 5:
                    name_field = fields[5]
                    patient_name = name_field.replace(self.COMP_SEP, " ").strip()

                logger.debug(
                    f"HL7 Patient: ID={patient_id}, Name={patient_name}"
                )
            except Exception as e:
                logger.warning(f"HL7 PID parse error: {e}")

        # ── OBR: Order Info ──
        elif segment_type == "OBR":
            try:
                # OBR-3: Filler Order Number (Sample ID)
                if len(fields) > 3:
                    sample_id = (
                        fields[3].split(self.COMP_SEP)[0].strip()
                    )

                # OBR-2 তেও Sample ID থাকতে পারে
                if not sample_id and len(fields) > 2:
                    sample_id = (
                        fields[2].split(self.COMP_SEP)[0].strip()
                    )

                logger.debug(f"HL7 Order: SampleID={sample_id}")
            except Exception as e:
                logger.warning(f"HL7 OBR parse error: {e}")

        # ── ORC: Common Order (কিছু analyzer ORC তে sample ID দেয়) ──
        elif segment_type == "ORC":
            try:
                if not sample_id and len(fields) > 3:
                    sample_id = (
                        fields[3].split(self.COMP_SEP)[0].strip()
                    )
            except Exception:
                pass

        # ── OBX: Observation Result (এটাই মূল data!) ──
        elif segment_type == "OBX":
            try:
                # OBX-3: Observation Identifier (Test Code)
                test_code = None
                if len(fields) > 3:
                    test_code = (
                        fields[3].split(self.COMP_SEP)[0].strip()
                    )
                    # কিছু HL7 এ test code OBX-3.2 তে থাকে
                    if not test_code and self.COMP_SEP in fields[3]:
                        parts = fields[3].split(self.COMP_SEP)
                        test_code = parts[1].strip() if len(parts) > 1 else parts[0].strip()

                # OBX-5: Observation Value (Result)
                result_value = None
                if len(fields) > 5:
                    result_value = (
                        fields[5].split(self.COMP_SEP)[0].strip()
                    )
```

```
                    # OBX-6: Units
                    result_unit = None
                    if len(fields) > 6:
                        result_unit = (
                            fields[6].split(self.COMP_SEP)[0].strip()
                        )

                    # OBX-7: Reference Range
                    ref_range = None
                    if len(fields) > 7:
                        ref_range = fields[7].strip()

                    # OBX-8: Abnormal Flags (H, L, HH, LL, N)
                    abnormal_flag = "N"
                    if len(fields) > 8:
                        abnormal_flag = fields[8].strip() or "N"

                    # OBX-2: Value Type (NM=Numeric, ST=String, CE=Coded)
                    # কিছু ক্ষেত্রে check করা দরকার
                    value_type = fields[2].strip() if len(fields) > 2 else "NM"

                    if test_code and result_value:
                        result = {
                            "analyzer_name": analyzer_name,
                            "sample_id": sample_id,
                            "patient_id": patient_id,
                            "patient_name": patient_name,
                            "test_code": test_code,
                            "result_value": result_value,
                            "result_unit": result_unit,
                            "ref_range": ref_range,
                            "abnormal_flag": abnormal_flag,
                            "result_datetime": datetime.now().strftime(
                                "%Y-%m-%d %H:%M:%S"
                            ),
                        }
                        results.append(result)

                except Exception as e:
                    logger.warning(f"HL7 OBX parse error: {e}")

        logger.info(
            f"📊 HL7 Parsed: {len(results)} results from {analyzer_name}"
        )
        return results
```

📄 Step 8: `core/ack_handler.py`

{} Python

```
import logging
from datetime import datetime

logger = logging.getLogger("LIS")


class ACKHandler:
    """
    Analyzer কে ACK (Acknowledgment) পাঠায়।
```

```
    ======================================
    Machine কে ACK না দিলে সে মনে করবে data পৌঁছায়নি
    এবং বারবার resend করবে — তাই এটা খুবই গুরুত্বপূর্ণ!
    """

    @staticmethod
    def build_astm_ack() -> bytes:
        """ASTM ACK = শুধু ACK character (0x06)"""
        return b"\x06"

    @staticmethod
    def build_astm_nak() -> bytes:
        """ASTM NAK = NAK character (0x15) — error হলে"""
        return b"\x15"

    @staticmethod
    def build_hl7_ack(raw_message: str) -> bytes:
        """
        HL7 ACK Message তৈরি করে
        Machine যে MSH পাঠিয়েছে সেটার উত্তরে ACK দিতে হবে
        """
        try:
            # MSH থেকে Sender/Receiver info বের করি
            lines = raw_message.replace("\x0b", "").replace("\x1c", "").strip()
            lines = lines.split("\r") if "\r" in lines else lines.split("\n")

            msh = lines[0] if lines else ""
            fields = msh.split("|")

            # MSH-3: Sending Application
            sending_app = fields[2] if len(fields) > 2 else "ANALYZER"
            # MSH-4: Sending Facility
            sending_fac = fields[3] if len(fields) > 3 else "LAB"
            # MSH-5: Receiving Application
            recv_app = fields[4] if len(fields) > 4 else "LIS"
            # MSH-6: Receiving Facility
            recv_fac = fields[5] if len(fields) > 5 else "HOSPITAL"
            # MSH-10: Message Control ID
            msg_id = fields[9] if len(fields) > 9 else "000"

            now = datetime.now().strftime("%Y%m%d%H%M%S")

            ack_msg = (
                f"MSH|^~\\&|{recv_app}|{recv_fac}|{sending_app}|{sending_fac}"
                f"|{now}||ACK|{msg_id}|P|2.4\r"
                f"MSA|AA|{msg_id}\r"
            )

            # MLLP framing যোগ করি
            return b"\x0b" + ack_msg.encode("utf-8") + b"\x1c\x0d"

        except Exception as e:
            logger.error(f"HL7 ACK build error: {e}")
            # Fallback generic ACK
            return (
                b"\x0bMSH|^~\\&|LIS|LAB|ANALYZER||"
                + datetime.now().strftime("%Y%m%d%H%M%S").encode()
                + b"||ACK|001|P|2.4\rMSA|AA|001\r\x1c\x0d"
            )
```

📄 Step 9: `listeners/tcp_listener.py`

{} Python

```python
import asyncio
import logging
from core.ack_handler import ACKHandler

logger = logging.getLogger("LIS")


class TCPListener:
    """
    TCP/LAN Listener
    ================
    Real Analyzer LAN cable দিয়ে data পাঠায় —
    এই listener সেই data receive করে।
    """

    def __init__(self, analyzer_config, parser, db_writer):
        self.name = analyzer_config["name"]
        self.host = analyzer_config.get("host", "0.0.0.0")
        self.port = analyzer_config["port"]
        self.protocol = analyzer_config["protocol"].upper()
        self.parser = parser
        self.db_writer = db_writer
        self.ack = ACKHandler()

    async def handle_client(self, reader, writer):
        addr = writer.get_extra_info("peername")
        logger.info(f"🔗 [{self.name}] Connected from {addr}")

        buffer = ""

        try:
            while True:
                data = await reader.read(8192)
                if not data:
                    break

                raw = data.decode("utf-8", errors="ignore")
                buffer += raw

                # ── ASTM: Message complete check (ETX = \x03) ──
                if self.protocol == "ASTM":
                    if "\x03" in buffer or "L|" in buffer:
                        # Complete message পেয়েছি
                        messages = buffer.split("\x03")
                        for msg in messages:
                            if msg.strip():
                                await self._process_message(msg)
                        buffer = ""
                        # ACK পাঠান
                        writer.write(self.ack.build_astm_ack())
                        await writer.drain()

                # ── HL7: Message complete check (MLLP End Block = \x1c) ──
                elif self.protocol == "HL7":
                    if "\x1c" in buffer or "\x0d" in buffer:
                        # MLLP message: \x0b ... \x1c\x0d
                        if "\x1c" in buffer:
                            parts = buffer.split("\x1c")
                            for msg in parts:
                                msg = msg.strip("\x0b").strip()
                                if msg and "MSH" in msg:
                                    await self._process_message(msg)
                            buffer = ""
```

```
                        # HL7 ACK পাঠান
                        ack_bytes = self.ack.build_hl7_ack(raw)
                        writer.write(ack_bytes)
                        await writer.drain()
                    else:
                        # Fallback: \x0d দিয়ে split
                        await self._process_message(buffer)
                        buffer = ""
                        ack_bytes = self.ack.build_hl7_ack(raw)
                        writer.write(ack_bytes)
                        await writer.drain()

                # ── Fallback: যদি কোনো delimiter না পাই ──
                else:
                    if len(buffer) > 500:
                        await self._process_message(buffer)
                        buffer = ""

        except asyncio.CancelledError:
            logger.info(f"🔌 [{self.name}] Connection cancelled")
        except ConnectionResetError:
            logger.warning(f"⚠️ [{self.name}] Connection reset by analyzer")
        except Exception as e:
            logger.error(f"❌ [{self.name}] Error: {e}")
        finally:
            try:
                writer.close()
                await writer.wait_closed()
            except Exception:
                pass
            logger.info(f"🔌 [{self.name}] Disconnected from {addr}")

    async def _process_message(self, raw_message: str):
        """Message parse করে DB তে save করে"""
        if not raw_message.strip():
            return

        logger.info(
            f"📩 [{self.name}] Raw ({len(raw_message)} chars): "
            f"{raw_message[:120]}..."
        )

        # Raw message log এ save
        self.db_writer.save_raw_message(self.name, raw_message)

        # Parse
        results = self.parser.parse(raw_message, self.name)

        # DB তে insert
        if results:
            self.db_writer.insert_results(results, raw_message)
        else:
            logger.warning(f"⚠️ [{self.name}] No results parsed from message")

    async def start(self):
        """TCP Server শুরু করে"""
        try:
            server = await asyncio.start_server(
                self.handle_client, self.host, self.port
            )
            logger.info(
                f"🚀 [{self.name}] TCP Listening on "
                f"{self.host}:{self.port} ({self.protocol})"
            )
            async with server:
                await server.serve_forever()
        except OSError as e:
```

```
            logger.error(
                f"❌ [{self.name}] Port {self.port} already in use! "
                f"Change port in config.yaml. Error: {e}"
            )
        except Exception as e:
            logger.error(f"❌ [{self.name}] TCP Start Error: {e}")
```

📄 Step 10: `listeners/serial_listener.py`

{} Python

```python
import serial
import time
import logging
from core.ack_handler import ACKHandler

logger = logging.getLogger("LIS")


class SerialListener:
    """
    Serial/RS-232 Listener
    ======================
    পুরনো Analyzer গুলো Serial cable দিয়ে connect থাকে।
    এই listener COM port থেকে data read করে।
    """

    def __init__(self, analyzer_config, parser, db_writer):
        self.name = analyzer_config["name"]
        self.port = analyzer_config.get("serial_port", "COM1")
        self.baud = analyzer_config.get("baud_rate", 9600)
        self.data_bits = analyzer_config.get("data_bits", 7)
        self.parity = analyzer_config.get("parity", "EVEN")
        self.stop_bits = analyzer_config.get("stop_bits", 1)
        self.protocol = analyzer_config["protocol"].upper()
        self.parser = parser
        self.db_writer = db_writer
        self.ack = ACKHandler()
        self.ser = None
        self.running = True

    def _get_parity(self):
        parity_map = {
            "NONE": serial.PARITY_NONE,
            "EVEN": serial.PARITY_EVEN,
            "ODD": serial.PARITY_ODD,
            "MARK": serial.PARITY_MARK,
            "SPACE": serial.PARITY_SPACE,
        }
        return parity_map.get(self.parity.upper(), serial.PARITY_EVEN)

    def _get_stopbits(self):
        stop_map = {
            1: serial.STOPBITS_ONE,
            1.5: serial.STOPBITS_ONE_POINT_FIVE,
            2: serial.STOPBITS_TWO,
        }
        return stop_map.get(self.stop_bits, serial.STOPBITS_ONE)

    def _get_bytesize(self):
```

```
        size_map = {5: serial.FIVEBITS, 6: serial.SIXBITS, 7: serial.SEVENBITS, 8:
serial.EIGHTBITS}
        return size_map.get(self.data_bits, serial.SEVENBITS)

    def start(self):
        """Serial port open করে continuously read করে"""
        try:
            self.ser = serial.Serial(
                port=self.port,
                baudrate=self.baud,
                bytesize=self._get_bytesize(),
                parity=self._get_parity(),
                stopbits=self._get_stopbits(),
                timeout=1,
            )
            logger.info(
                f"🚀 [{self.name}] Serial Listening on {self.port} "
                f"({self.baud},{self.data_bits},{self.parity},{self.stop_bits}) "
                f"({self.protocol})"
            )
        except serial.SerialException as e:
            logger.error(
                f"❌ [{self.name}] Cannot open {self.port}: {e}\n"
                f"   → Check: Is cable connected? Is port correct in config.yaml?"
            )
            return

        buffer = ""

        while self.running:
            try:
                raw = self.ser.read(1024).decode("utf-8", errors="ignore")

                if raw:
                    buffer += raw

                    # ── ASTM: ETX (\x03) পেলে message complete ──
                    if self.protocol == "ASTM" and "\x03" in buffer:
                        messages = buffer.split("\x03")
                        for msg in messages[:-1]:
                            if msg.strip():
                                self._process_message(msg)
                        buffer = messages[-1]

                        # ACK পাঠান
                        self.ser.write(self.ack.build_astm_ack())
                        logger.debug(f"📨 [{self.name}] ASTM ACK sent")

                    # ── HL7: End Block (\x1c) পেলে message complete ──
                    elif self.protocol == "HL7" and "\x1c" in buffer:
                        parts = buffer.split("\x1c")
                        for msg in parts[:-1]:
                            msg = msg.strip("\x0b").strip()
                            if msg and "MSH" in msg:
                                self._process_message(msg)
                        buffer = parts[-1]

                        # HL7 ACK
                        ack = self.ack.build_hl7_ack(raw)
                        self.ser.write(ack)
                        logger.debug(f"📨 [{self.name}] HL7 ACK sent")

                    # ── Buffer overflow protection ──
                    elif len(buffer) > 10000:
                        self._process_message(buffer)
                        buffer = ""
```

```
            except serial.SerialException as e:
                logger.error(f"❌ [{self.name}] Serial Error: {e}")
                time.sleep(5)
                # Reconnect try
                try:
                    self.ser.close()
                    self.ser.open()
                    logger.info(f"🔌 [{self.name}] Serial Reconnected")
                except Exception:
                    pass

            except Exception as e:
                logger.error(f"❌ [{self.name}] Error: {e}")
                time.sleep(1)

    def _process_message(self, raw_message: str):
        if not raw_message.strip():
            return

        logger.info(
            f"📩 [{self.name}] Serial Raw ({len(raw_message)} chars): "
            f"{raw_message[:120]}..."
        )

        self.db_writer.save_raw_message(self.name, raw_message)
        results = self.parser.parse(raw_message, self.name)

        if results:
            self.db_writer.insert_results(results, raw_message)

    def stop(self):
        self.running = False
        if self.ser and self.ser.is_open:
            self.ser.close()
            logger.info(f"🔒 [{self.name}] Serial Port Closed")
```

📄 Step 11: `main.py` (Entry Point)

{} Python

```
import asyncio
import threading
import yaml
import logging
import os
import sys
from datetime import datetime

from database.db_connection import OracleDB
from database.db_writer import DBWriter
from parsers.astm_parser import ASTMPParser
from parsers.hl7_parser import HL7Parser
from listeners.tcp_listener import TCPListener
from listeners.serial_listener import SerialListener


# ============================================================
# LOGGING SETUP
# ============================================================
def setup_logging():
    os.makedirs("logs", exist_ok=True)
```

```
    log_file = f"logs/lis_{datetime.now().strftime('%Y%m%d')}.log"

    logging.basicConfig(
        level=logging.INFO,
        format="%(asctime)s | %(levelname)-7s | %(message)s",
        datefmt="%Y-%m-%d %H:%M:%S",
        handlers=[
            logging.FileHandler(log_file, encoding="utf-8"),
            logging.StreamHandler(sys.stdout),
        ],
    )
    return logging.getLogger("LIS")


# ============================================================
# MAIN
# ============================================================
def main():
    logger = setup_logging()

    print("=" * 60)
    print("  🖥️  LIS MIDDLEWARE - Lab to Oracle DB Connector")
    print("  📡  Waiting for Analyzer Data...")
    print("=" * 60)

    # ── Config Load ──
    config_path = "config/config.yaml"
    if not os.path.exists(config_path):
        logger.error(f"❌ Config file not found: {config_path}")
        logger.error("   → config/config.yaml তৈরি করুন!")
        return

    with open(config_path, "r", encoding="utf-8") as f:
        config = yaml.safe_load(f)

    analyzers = config.get("analyzers", [])
    if not analyzers:
        logger.error("❌ config.yaml এ কোনো analyzer নেই!")
        return

    logger.info(f"📋 Found {len(analyzers)} analyzer(s) in config")

    # ── DB Connect ──
    db = OracleDB(config_path)
    try:
        db.connect()
    except Exception:
        logger.error("❌ DB Connection Failed! Check config.yaml")
        logger.error("   → host, port, service_name, username, password ঠিক আছে?")
        return

    db_writer = DBWriter(db)

    # ── Parsers ──
    astm_parser = ASTMPParser()
    hl7_parser = HL7Parser()

    # ── Start Listeners ──
    loop = asyncio.new_event_loop()
    serial_threads = []

    for az in analyzers:
        name = az["name"]
        protocol = az.get("protocol", "ASTM").upper()
        conn_type = az.get("connection_type", "TCP").upper()
```

```
            # Parser select
            parser = astm_parser if protocol == "ASTM" else hl7_parser

            if conn_type == "TCP":
                listener = TCPListener(az, parser, db_writer)
                loop.create_task(listener.start())
                logger.info(
                    f"  ✅ {name:25s} | {protocol:4s} | TCP  | "
                    f"Port {az.get('port', '?')}"
                )

            elif conn_type == "SERIAL":
                listener = SerialListener(az, parser, db_writer)
                thread = threading.Thread(
                    target=listener.start, name=name, daemon=True
                )
                thread.start()
                serial_threads.append((listener, thread))
                logger.info(
                    f"  ✅ {name:25s} | {protocol:4s} | Serial | "
                    f"{az.get('serial_port', '?')}"
                )

            else:
                logger.warning(f"  ⚠️ {name}: Unknown connection_type '{conn_type}'")

        print("=" * 60)
        logger.info("🏥 LIS Middleware is RUNNING!")
        logger.info("   Press Ctrl+C to stop")
        print("=" * 60)

        # ── Run ──
        try:
            loop.run_forever()
        except KeyboardInterrupt:
            logger.info("🔴 Shutting down...")
        finally:
            # Cleanup
            for listener, thread in serial_threads:
                listener.stop()
            loop.close()
            db.close()
            logger.info("👋 LIS Middleware Stopped")


if __name__ == "__main__":
    main()
```

📄 Step 12: `__init__.py` Files (সব ফাঁকা)

এই ফাইলগুলো তৈরি করুন (ভেতরে কিছু লেখার দরকার নেই):

{} PowerShell

```
New-Item -Path "listeners\__init__.py" -ItemType File -Force
New-Item -Path "parsers\__init__.py" -ItemType File -Force
New-Item -Path "database\__init__.py" -ItemType File -Force
New-Item -Path "core\__init__.py" -ItemType File -Force
```

🗄️ Step 13: Oracle DB Tables (SQL Script)

এটা Oracle SQL Developer বা SQL*Plus এ run করুন:

{} SQL

```sql
-- ============================================================
-- LIS MIDDLEWARE - Oracle DB Tables
-- ============================================================

-- 1. Main Results Table
CREATE TABLE lab_results (
    result_id       NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    sample_id       VARCHAR2(50)  NOT NULL,
    patient_id      VARCHAR2(50),
    patient_name    VARCHAR2(200),
    test_code       VARCHAR2(30)  NOT NULL,
    result_value    VARCHAR2(100),
    result_unit     VARCHAR2(30),
    ref_range       VARCHAR2(50),
    abnormal_flag   VARCHAR2(5)   DEFAULT 'N',
    analyzer_name   VARCHAR2(50),
    result_datetime DATE          DEFAULT SYSDATE,
    raw_message     CLOB,
    status          VARCHAR2(10)  DEFAULT 'NEW',
    verified_by     VARCHAR2(50),
    verified_at     DATE
);

-- 2. Raw Message Log (Debugging এর জন্য)
CREATE TABLE lis_message_log (
    log_id          NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    analyzer_name   VARCHAR2(50),
    raw_message     CLOB,
    received_at     DATE DEFAULT SYSDATE
);

-- 3. Indexes (APEX এ Fast Search)
CREATE INDEX idx_lab_sample   ON lab_results(sample_id);
CREATE INDEX idx_lab_patient  ON lab_results(patient_id);
CREATE INDEX idx_lab_datetime ON lab_results(result_datetime);
CREATE INDEX idx_lab_analyzer ON lab_results(analyzer_name);
CREATE INDEX idx_lab_status   ON lab_results(status);

-- 4. Auto-cleanup old logs (30 দিন পর পুরনো log মুছে যাবে)
-- (Optional - DBMS_SCHEDULER দিয়ে)

COMMENT ON TABLE lab_results IS 'LIS Middleware - Lab Test Results';
COMMENT ON TABLE lis_message_log IS 'LIS Middleware - Raw Message Log';
```

🚀 Step 14: Run করুন!

{} PowerShell

```powershell
cd F:\Nadim\lis-middleware
```

```
# Config edit করুন (DB IP, password, etc.)
notepad config\config.yaml

# Run!
python main.py
```

Expected Output:

{} text

```
============================================================
  🏥 LIS MIDDLEWARE - Lab to Oracle DB Connector
  📡 Waiting for Analyzer Data...
============================================================
2025-01-15 10:00:00 | INFO     | 📋 Found 5 analyzer(s) in config
2025-01-15 10:00:01 | INFO     | ✅ Oracle DB Connected: 192.168.1.100:1521/ORCL
2025-01-15 10:00:01 | INFO     |    ✅ Sysmex_XN_1000          | ASTM | TCP    | Port 6661
2025-01-15 10:00:01 | INFO     |    ✅ Mindray_BS240           | HL7  | TCP    | Port 6662
2025-01-15 10:00:01 | INFO     |    ✅ Roche_Cobas_c311        | ASTM | Serial | COM3
2025-01-15 10:00:01 | INFO     |    ✅ Celltac_MEK9100         | ASTM | TCP    | Port 6663
2025-01-15 10:00:01 | INFO     |    ✅ Snibe_Maglumi_800       | HL7  | TCP    | Port 6664
============================================================
2025-01-15 10:00:01 | INFO     | 🏥 LIS Middleware is RUNNING!
```

📌 নতুন Machine যোগ করার নিয়ম

শুধু `config/config.yaml` এ নতুন entry দিন — কোড এ হাত দেবেন না!

{} YAML

```yaml
  # নতুন Machine:
  - name: "Abbott_CellDyn_1800"
    brand: "Abbott"
    model: "Cell-Dyn 1800"
    protocol: "ASTM"            # Machine এর manual দেখুন
    connection_type: "TCP"      # LAN নাকি Serial
    host: "0.0.0.0"
    port: 6666                  # নতুন unused port দিন
```

ব্যস! `python main.py` restart দিন — নতুন machine ready! 🎉

⚠️ Troubleshooting

সমস্যা	সমাধান
`Port already in use`	config.yaml এ port change করুন

`Cannot open COM3`	Serial cable check করুন, Device Manager এ COM port দেখুন
`DB Connection Failed`	DB IP, port, username, password check করুন
`No results parsed`	Analyzer এর protocol (ASTM/HL7) ঠিক আছে কিনা দেখুন
`oracledb not found`	`pip install oracledb` করুন

কোনো সমস্যা হলে log file দেখুন: `E:\Nadim\lis_middleware\logs\`

